

Урок 4. Корутины. Continuation, возврат значений.

28 ноября 2019

В этом уроке рассмотрим более подробно, как Continuation обрабатывает ответ от suspend функций. А также, как корутина может вернуть какое-либо значение, как результат своей работы.

В первом уроке про Continuation мы разобрались, как происходит вызов suspend функций и получение от них результата. Есть еще пара интересных моментов в реализации Continuation, про которые я хотел бы рассказать. Это возврат значения корутиной и обработка ответа от suspend функций.

По ходу объяснения мы продолжим добавлять логику в рассматриваемый код Continuation и в итоге получим очень близкий к реальному вариант. После этого урока вы будете иметь практически полное представление о Continuation.

Этот урок не является обязательным для понимания дальнейшего материала курса.

Подробнее

Возврат значения корутиной

Некоторые типы корутин могут возвращать какое-либо значение, как результат своей работы. Давайте подробно рассмотрим, как это реализовано в Continuation. Но сначала немного изменим наше упрощенное представление о методе Continuation.invokeSuspend:

```
1 void invokeSuspend(Object result) {  
2     ...  
3 }
```

Во-первых, он не void, а возвращает Object значение. Это нужно, чтобы корутина могла вернуть результат своей работы.

Во-вторых, для упрощения объяснений я рассказывал, что suspend функции вызывают метод invokeSuspend. Но на самом деле (и как мы могли видеть в прошлом уроке) suspend функции вызывают метод Continuation.resume. В отличие от invokeSuspend, этот метод не генерируется, а содержится в базовом для Continuation классе.

В методе resume содержится логика, которая вызывает invokeSuspend и проверяет результат его работы. В зависимости от того, что вернет invokeSuspend, корутина определяет, что Continuation либо завершил одну из веток свитча и ждет suspend функцию, либо полностью завершил свою работу и корутина может быть завершена.

Очень упрощенно метод resume выглядит так:

```
1 void resume(Object result) {  
2     Object outcome = invokeSuspend(result);  
3     if (outcome == COROUTINE_SUSPENDED) return;  
4     completeCoroutine(outcome);  
5 }
```

Когда suspend функция вызывает resume и передает ему результат своей работы, resume в свою очередь вызывает invokeSuspend, передает ему значение, которое пришло из suspend функции, и проверяет результат. Варианта тут только два:

1) invokeSuspend вернул специальную константу COROUTINE_SUSPENDED. Это значит, что он запустил suspend функцию, и на этом метод resume завершается без каких либо действий. Когда suspend функция выполнит свою работу, она снова вызовет метод resume, а он в свою очередь вызовет invokeSuspend. Так и бегает по кругу пока корутина не завершится.

2) invokeSuspend вернул какое-то значение, отличное от COROUTINE_SUSPENDED. Для метода resume это значение является результатом работы корутины. Метод resume начинает процедуру завершения корутины.

Осталось понять, как invokeSuspend может что-либо вернуть. Как я уже сказал, ранее рассмотренные мною версии этого метода были упрощены и ничего не возвращали. Пришло время взглянуть правде в глаза)

Возьмем все тот же пример с загрузкой файла, но поменяем билдер корутины на async

```
1 async {  
2     val url = buildUrl()  
3  
4     val file = download(url) // suspend function  
5  
6     toast("Url is downloaded: $url")  
7  
8     val size = unzip(file) // suspend function  
9  
10    toast("File is unzipped: $size")  
11  
12    size  
13 }
```

Сейчас мы не будем вникать в чем разница между корутинами, которые создаются билдерами launch и async. Мы еще поговорим об этом позже. Если вкратце, то от async корутины мы можем получить результат ее работы.

Теперь корутина может вернуть значение. Будем возвращать результат функции unzip - размер распакованного содержимого.

Переходим на следующий уровень сложности кода Continuation:

```

1  Object invokeSuspend(Object result) {
2      switch (label) {
3          case 0: {
4              url = buildUrl();
5              label = 1;
6
7              download(url, this);
8              return COROUTINE_SUSPENDED;
9          }
10         case 1: {
11             file = (File) result;
12             toast("Url is downloaded: " + url);
13             label = 2;
14
15             unzip(file, this);
16             return COROUTINE_SUSPENDED;
17         }
18         case 2: {
19             size = (Long) result;
20             toast("File is unzipped, size: " + size);
21             return size;
22         }
23     }
24
25     throw new IllegalStateException("...");
26 }

```

Давайте пройдемся по ключевым изменениям в методе invokeSuspend.

Метод invokeSuspend теперь не void, а возвращает Object. Это нужно, чтобы invokeSuspend мог вернуть значение в resume. Как мы помним, это может быть либо COROUTINE_SUSPENDED, либо значение, которое будет являться результатом работы корутины.

В каждой ветке свитча, где запускается suspend функция, invokeSuspend возвращает COROUTINE_SUSPENDED. Для resume это сигнал о том, что у Continuation еще есть невыполненные части кода и надо ждать, пока suspend функция отработает и продолжит выполнение Continuation.

А в последней ветке свитча мы возвращаем реальное значение - size. Для resume - это сигнал того, что Continuation выполнил все части кода и корутина может быть завершена. Он стартует процедуру завершения.

Если вдруг логика выходит за пределы свитча, то выбрасывается исключение. Такого по идее не должно произойти.

Suspend функции с готовым результатом

В прошлом уроке про suspend функции мы узнали, что suspend функция может отработать как синхронно (если результат сразу готов), так и асинхронно (если нужна асинхронная работа).

Когда Continuation вызывает suspend функцию, то могут быть два варианта развития событий..

- 1) Если у suspend функции сразу готов результат, то она его и вернет. Прямо в место вызова функции, а не через вызов метода invokeSuspend.
- 2) Если же suspend функции нужно время для работы, она возвращает константу COROUTINE_SUSPENDED, начинает асинхронную работу и результат возвращает через вызов метода invokeSuspend.

В следующем уроке мы увидим, как это у нее получается. Это действительно неочевидно, особенно с учетом того факта, что и в синхронном и в асинхронном случаях вызывается метод continuation.resume, который вроде как должен вести в invokeSuspend.

А пока вернемся к Continuation. Давайте посмотрим, как он с этим справляется.

```

1  Object invokeSuspend(Object result) {
2      switch (label) {
3          case 0: {
4              url = buildUrl();
5              label = 1;
6
7              result = download(url, this);
8              if (result == COROUTINE_SUSPENDED) return COROUTINE_SUSPENDED;
9          }
10         case 1: {
11             file = (File) result;
12             toast("Url is downloaded: " + url);
13             label = 2;
14
15             result = unzip(file, this);
16             if (result == COROUTINE_SUSPENDED) return COROUTINE_SUSPENDED;
17         }
18         case 2: {
19             size = (Long) result;
20             toast("File is unzipped, size: " + size);
21             return size;
22         }
23     }
24
25     throw new IllegalStateException("...");
26 }

```

Вызов suspend функций теперь выглядит так:

```

1  result = download(url, this);
2  if (result == COROUTINE_SUSPENDED) return COROUTINE_SUSPENDED;

```

Мы проверяем результат вызова suspend функции.

Если результат - COROUTINE_SUSPENDED, значит функция ушла в асинхронный запуск и позже сама вызовет Continuation.resume. А сейчас invokeSuspend завершает свою работу и возвращает COROUTINE_SUSPENDED. Метод resume получает эту константу как результат вызова invokeSuspend, понимает, что корутина еще не завершена, и пока ничего не делает.

Если же из `download` нам пришло реальное значение, значит `suspend` функция отработала сразу, без асинхронщины. Это дает нам возможность не выходить из метода `invokeSuspend`, а просто перейти в следующую ветку свитча. Тем более, что результат выполнения `suspend` функции у нас уже лежит в переменной `result`.

```
1    ...
2    result = download(url, this);
3    if (result == COROUTINE_SUSPENDED) return COROUTINE_SUSPENDED;
4  }
5  case 1: {
6    file = (File) result;
7    ...
```

Если в конце `case` нет `return`, то выполнение пойдет в следующий `case`. В нашем случае он пойдет в `case 1` и преобразует `result`, полученный из `download`, к типу `File`.

Т.е. все сработает точно так же, как если бы `suspend` функция отработала асинхронно, через `resume` вызвала метод `invokeSuspend` и передала туда значение, а `switch` направил нас в эту ветку. Просто переход к следующей части кода произойдет в рамках текущего выполнения `invokeSuspend`, без участия `suspend` функции и последующего нового вызова `invokeSuspend`.

На этом закончим тему `Continuation`. Мы разобрали его очень подробно. В следующем уроке рассмотрим, как работает под капотом `suspend` функция. После этого у вас сложится полная картина, как корутина и `suspend` функции взаимодействуют между собой.

Кратко

Возврат значения корутиной.

Некоторые типы корутин могут возвращать какое-либо значение, как результат своей работы. Давайте подробно рассмотрим, как это происходит в `Continuation`.

Возьмем все тот же пример, но поменяем билдер корутины на `async`

```

1  async {
2      val url = buildUrl()
3
4      val file = download(url) // suspend function
5
6      toast("File is downloaded: $url")
7
8      val size = unzip(file) // suspend function
9
10     toast("File is unzipped, size = $size")
11
12     size
13 }

```

Сейчас мы не будем вникать в чем разница между корутинами, которые создаются билдерами `launch` и `async`. Если вкратце, то от `async` корутины мы можем получить результат ее работы.

Теперь корутина может вернуть значение. Будем возвращать результат функции `unzip` - размер распакованного содержимого.

Смотрим код `Continuation`, теперь я не убирал из него логику возврата значения:

```

1  Object invokeSuspend(Object result) {
2      switch (label) {
3          case 0: {
4              url = buildUrl();
5              label = 1;
6
7              download(url, this);
8              return COROUTINE_SUSPENDED;
9          }
10         case 1: {
11             file = (File) result;
12             toast("File is downloaded: " + url);
13             label = 2;
14
15             unzip(file, this);
16             return COROUTINE_SUSPENDED;
17         }
18         case 2: {
19             size = (Long) result;
20             toast("File is unzipped, size: " + size);
21             return size;
22         }
23     }
24     throw new IllegalStateException("...");
25 }
26

```

Давайте пройдемся по ключевым изменениям. Но перед этим небольшое отступление.

Для упрощения объяснений я рассказывал, что `suspend` функции вызывают метод `invokeSuspend`, чтобы продолжить выполнение корутины. Но на самом деле `suspend` функции вызывают метод `Continuation.resume`, как мы видели в прошлом уроке. А в методе `resume` содержится логика, которая вызывает `invokeSuspend`, проверяет результат его работы и в зависимости от него выполняет определенные действия.

Разбираем код.

Метод `invokeSuspend` теперь не `void`, а возвращает `Object`. Это нужно, чтобы `invokeSuspend` мог вернуть значение в `resume`. Это может быть либо специальная константа `COROUTINE_SUSPENDED`, либо значение, которое будет являться результатом работы корутины.

В каждой ветке свитча, где запускается `suspend` функция, `invokeSuspend` возвращает `COROUTINE_SUSPENDED`. Для `resume` это сигнал о том, что `Continuation` запустил `suspend` функцию и надо ждать, пока она отработает и продолжит выполнение `Continuation`.

А в последней ветке свитча мы возвращаем не `COROUTINE_SUSPENDED`, а реальное значение - `size`. Для `resume` - это сигнал того, что `Continuation` выполнил все части кода и корутина может быть завершена. Он стартует процедуру завершения.

Если вдруг логика выходит за пределы свитча, то выбрасывается исключение, т.к. такого по идее не должно произойти.

Suspend функции с готовым результатом

На прошлом уроке мы разбирали, что `suspend` функция может отработать как асинхронно, так и синхронно. В следующем уроке мы подробно рассмотрим, что при этом происходит под капотом. Пока же нам необходимо знать следующее. Если у `suspend` функции сразу готов результат, она его возвращает. Если же результат не готов, она возвращает константу `COROUTINE_SUSPENDED` и начинает асинхронную работу.

Давайте посмотрим, как это обрабатывается в `Continuation`. Я не убирал из него логику получения синхронного ответа от `suspend` функций.

```

1  Object invokeSuspend(Object result) {
2      switch (label) {
3          case 0: {
4              url = buildUrl();
5              label = 1;
6
7              result = download(url, this);
8              if (result == COROUTINE_SUSPENDED) return COROUTINE_SUSPENDED;
9          }
10         case 1: {
11             file = (File) result;
12             toast("Url is downloaded: " + url);
13             label = 2;
14
15             result = unzip(file, this);
16             if (result == COROUTINE_SUSPENDED) return COROUTINE_SUSPENDED;
17         }
18         case 2: {
19             size = (Long) result;
20             toast("File is unzipped, size: " + size);
21             return size;
22         }
23     }
24
25     throw new IllegalStateException("...");
26 }

```

При вызове suspend функции мы проверяем результат вызова.

Если результат - COROUTINE_SUSPENDED, значит функция ушла в асинхронный запуск и позже сама вызовет Continuation.resume. А invokeSuspend завершает свою работу и возвращает COROUTINE_SUSPENDED в resume. Метод resume получает эту константу как результат вызова invokeSuspend, понимает, что корутина еще не завершена, и пока ничего не делает.

Если же из suspend функции нам пришло реальное значение, значит suspend функция отработала сразу, без асинхронщины. Это дает нам возможность не выходить из метода invokeSuspend, а просто перейти в следующую ветку свитча. Тем более, что результат выполнения suspend функции попадает в переменную result.